Q1 Employee Record Management

CODE:

```
import java.io.*;

import java.util.Scanner;

public class EmployeeRecordManagement {

    private static final String FILE_NAME = "employees.txt";

    // Method to add an employee record using character streams (FileWriter & BufferedWriter)

    public static void addEmployee(Scanner sc) {

        System.out.print("Enter Employee ID: ");

        String id = sc.nextLine().trim();

        System.out.print("Enter Name: ");

        String name = sc.nextLine().trim();

        System.out.print("Enter Department: ");

        String dept = sc.nextLine().trim();

        System.out.print("Enter Salary: ");

        double salary = Double.parseDouble(sc.nextLine().trim());

        try (BufferedWriter bw = new BufferedWriter(new FileWriter(FILE_NAME, true))) {

            bw.write(id + "," + name + "," + dept + "," + salary);

            bw.newLine();

            System.out.println("Employee record added successfully");

        } catch (IOException e) {

            System.out.println("Error writing to file: " + e.getMessage());

        }

    }
```

```
    // Method to display all employee records using character streams (FileReader &
BufferedReader)
    public static void displayAllEmployees() {
        File file = new File(FILE_NAME);
        if (!file.exists()) {
            System.out.println("File not found or no records available.");
            return;
        }
        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
            boolean hasRecords = false;
            System.out.println("\n--- Stored Employee Records ---");
            while ((line = br.readLine()) != null) {
                if (!line.trim().isEmpty()) {
                    String[] data = line.split(",");
                    if (data.length == 4) {
                        System.out.println("ID: " + data[0] + " | Name: " + data[1] +
                                           " | Department: " + data[2] + " | Salary: " + data[3]);
                        hasRecords = true;
                    }
                }
            }
            if (!hasRecords) {
                System.out.println("No employee records found.");
            }
```

```java
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }
}

// Method to search for an employee by ID
public static void searchEmployee(Scanner sc) {
    File file = new File(FILE_NAME);
    if (!file.exists()) {
        System.out.println("File not found or no records available.");
        return;
    }

    System.out.print("Enter Employee ID to search: ");
    String searchId = sc.nextLine().trim();
    boolean found = false;

    try (BufferedReader br = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] data = line.split(",");
            if (data.length == 4 && data[0].equalsIgnoreCase(searchId)) {
                System.out.println("\nRecord Found:");
                System.out.println("Employee ID : " + data[0]);
                System.out.println("Name        : " + data[1]);
```

```
                System.out.println("Department  : " + data[2]);
                System.out.println("Salary      : " + data[3]);
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Record not found for Employee ID: " + searchId);
        }
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    while (true) {
        System.out.println("\n=== Employee Record Management ===");
        System.out.println("1. Add Employee");
        System.out.println("2. Display All Employees");
        System.out.println("3. Search Employee");
        System.out.println("4. Exit");
        System.out.print("Choose an option: ");
```

```java
            if (!sc.hasNextLine()) break;
            String choice = sc.nextLine().trim();

            switch (choice) {
                case "1":
                    addEmployee(sc);
                    break;
                case "2":
                    displayAllEmployees();
                    break;
                case "3":
                    searchEmployee(sc);
                    break;
                case "4":
                    System.out.println("Exiting application.");
                    sc.close();
                    return;
                default:
                    System.out.println("Invalid choice. Please select again.");
            }
        }
    }
}
```

C:\Windows\System32\cmd.e

```
=== Employee Record Management ===
1. Add Employee
2. Display All Employees
3. Search Employee
4. Exit
Choose an option: 1
Enter Employee ID: E101
Enter Name: Arun
Enter Department: IT
Enter Salary: 35000
Employee record added successfully

=== Employee Record Management ===
1. Add Employee
2. Display All Employees
3. Search Employee
4. Exit
Choose an option: 1
Enter Employee ID: E102
Enter Name: Priya
Enter Department: HR
Enter Salary: 42000
Employee record added successfully

=== Employee Record Management ===
1. Add Employee
2. Display All Employees
3. Search Employee
4. Exit
Choose an option: 2

--- Stored Employee Records ---
ID: E101 | Name: Arun | Department: IT | Salary: 35000.0
ID: E102 | Name: Priya | Department: HR | Salary: 42000.0
```